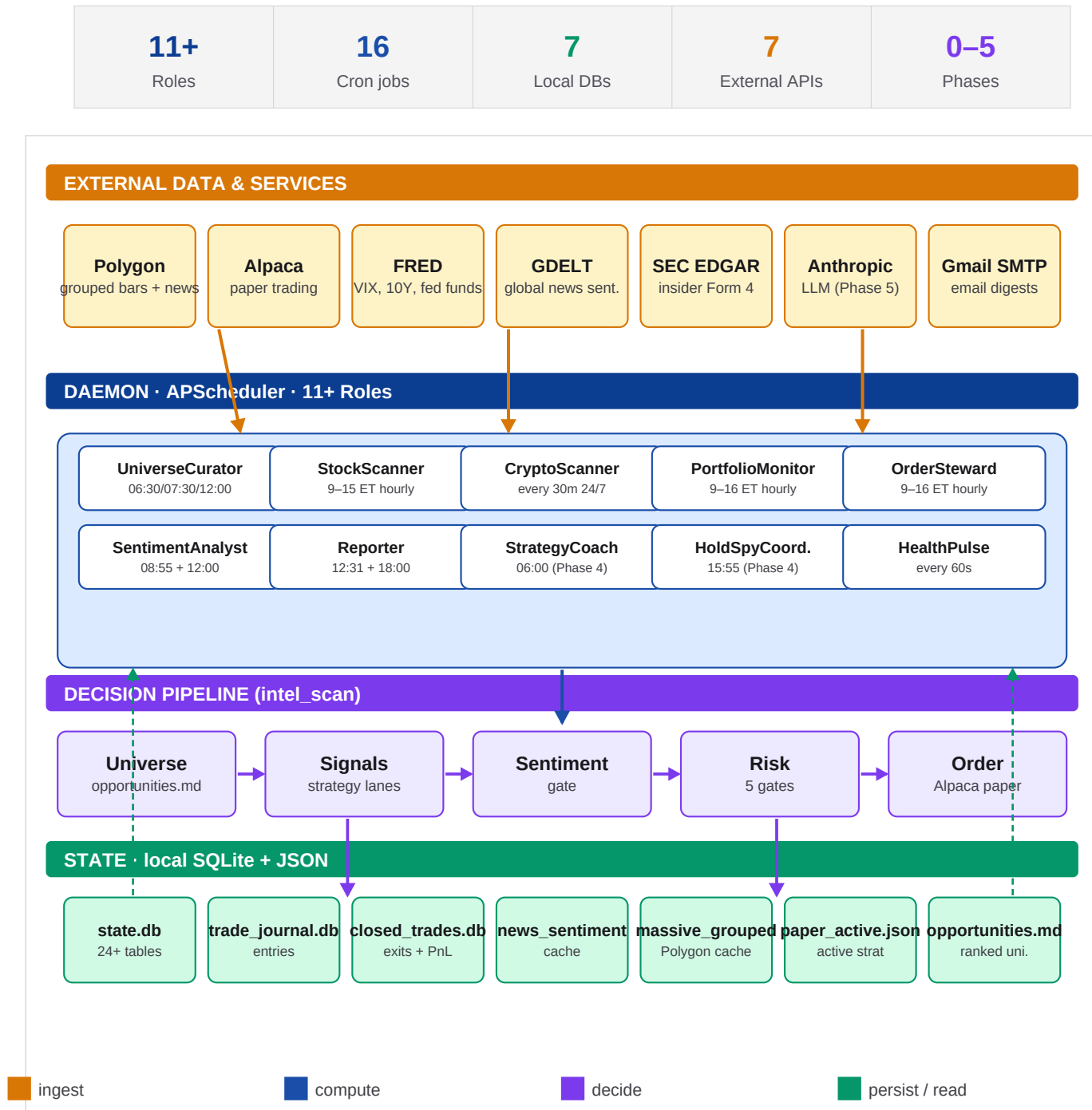


Trading Bot

End-to-End Architecture & Trade Lifecycle



System architecture — orange = ingest · blue = compute · purple = decide · green = persist

Table of Contents

- | | |
|-----|--|
| 1. | Executive Summary |
| 2. | Top-Level Layout |
| 3. | Subsystems & Roles |
| 4. | The Trade Lifecycle — with flow diagram |
| 5. | Decision Flow inside intel_scan — with branching diagram |
| 6. | Risk Gate — funnel diagram |
| 7. | Scheduling — daily timeline visual |
| 8. | Regime-Driven Allocation — bar chart |
| 9. | External Integrations & APIs |
| 10. | Configuration, Secrets & Safety Rails |
| 11. | State Stores — who writes, who reads |
| 12. | Lab Self-Improvement Loop — cyclical diagram |
| 13. | Operations & Failure Modes |
| 14. | Worked Example — one trade end-to-end |
| 15. | Quick Reference |

1. Executive Summary

The Trading Bot is a phase-gated, role-based algorithmic trading system running on Alpaca paper trading. It ingests market data from Polygon, macro context from FRED/GDELT/SEC, ranks a universe in two stages (composite score → strategy lanes), evaluates entries hourly during market hours, places orders through Alpaca with mandatory stop-losses, and reconciles fills end-of-day to a closed-trades ledger that feeds a self-improving Lab pipeline.

All state is local SQLite — no external database. The live universe is snapshotted to a human-readable Markdown file `strategy/opportunities.md` that downstream consumers re-read each cycle.

■ Core Pipeline

Polygon grouped → Stage-1 (top-100) → Stage-2 lanes → **opportunities.md** → intel_scan (hourly) → strategy signal → sentiment gate → 5-gate risk → Alpaca order → trade_journal → reconciliation → closed_trades → Lab walk-forward → promotion → new **paper_active.json**

✓ OK

Paper-only enforcement. The Alpaca client refuses to start unless `ALPACA_BASE_URL` begins with `https://paper-api.alpaca.markets`. `BOT_MODE=live` raises `LiveModeDisabled`; the live launchd plist exists but is not loaded.

! CAUTION

Manual kill switch. Touch `data/pause.flag` to halt all order-placing jobs without affecting monitoring jobs. No daemon restart required.

* NOTE

Auto-fallback to SPY. If 30-day paper alpha falls below SPY, the StrategyCoach flips a flag and `HoldSpyCoordinator` drives a 5-day exit / SPY transition.

2. Top-Level Layout

Repository root: /Users/bharathkandala/Trading

```
Trading/
■■■ src/trading_bot/      # All Python source (single package)
■   ■■■ cli.py           # Click CLI – every subcommand
■   ■■■ daemon.py        # Long-running daemon entrypoint
■   ■■■ scheduler_jobs.py # APScheduler job registration (timing source)
■   ■■■ orchestrator.py  # Trade decision loop
■   ■■■ alpaca_client.py  # Paper-only Alpaca API wrapper
■   ■■■ massive_client.py # Polygon REST client
■   ■■■ screener.py       # Stage-1 score, Stage-2 merge
■   ■■■ strategy_lanes.py # momentum / mean-reversion / breakout
■   ■■■ strategy.py       # Per-regime signal generation
■   ■■■ risk_manager.py   # 5-gate risk check
■   ■■■ regime.py         # Market regime classifier
■   ■■■ intelligence.py   # FRED + GDELT + SEC + Alpaca news
■   ■■■ news_sentiment.py # Cached Polygon sentiment
■   ■■■ trade_journal.py  # Append-only trade log
■   ■■■ reconciliation.py  # Closed-trade tracker
■   ■■■ pnl_state.py      # Daily/weekly P&L aggregation
■   ■■■ state_db.py       # SQLAlchemy models (24+ tables)
■   ■■■ lab.py            # Self-improvement loop
■   ■■■ walkforward.py    # Walk-forward backtest engine
■   ■■■ reports.py        # HTML email builders
■   ■■■ dashboard/       # FastAPI + HTMX dashboard
■   ■■■ backtest/        # Bar storage + simulator + metrics
■   ■■■ roles/           # 11+ Role objects (the actor model)
■■■ strategy/            # Live, mostly-human-readable config
■■■ data/                # Runtime state (SQLite + JSON)
■■■ ops/launchd/         # macOS plists (daemon, lab, supervisor)
■■■ migrations/          # Alembic auto-applied at boot
■■■ runs/                # Structured per-role JSON logs
■■■ tests/ · docs/
■■■ pyproject.toml · uv.lock · README.md · .env (uncommitted)
```

Five Entry Points

#	Entry	Invocation	Role
1	CLI	bot <subcommand> via cli.py	Manual ops / debugging
2	Daemon	python -m trading_bot.daemon	Live trading scheduler
3	Lab	python -m trading_bot.lab	Walk-forward search
4	Supervisor	ops/launchd/supervisor plist	Health monitor / alerts
5	Dashboard	python -m trading_bot.dashboard.app	FastAPI + HTMX UI

3. Subsystems & Roles

The daemon doesn't run a monolithic loop — it instantiates a set of **Role** objects (in `src/trading_bot/roles/`), each inheriting from `BaseRole` with a `safe_run(ctx)` wrapper that catches exceptions, logs structured JSON to `runs/`, and records latency to `state.db::RoleRun`.

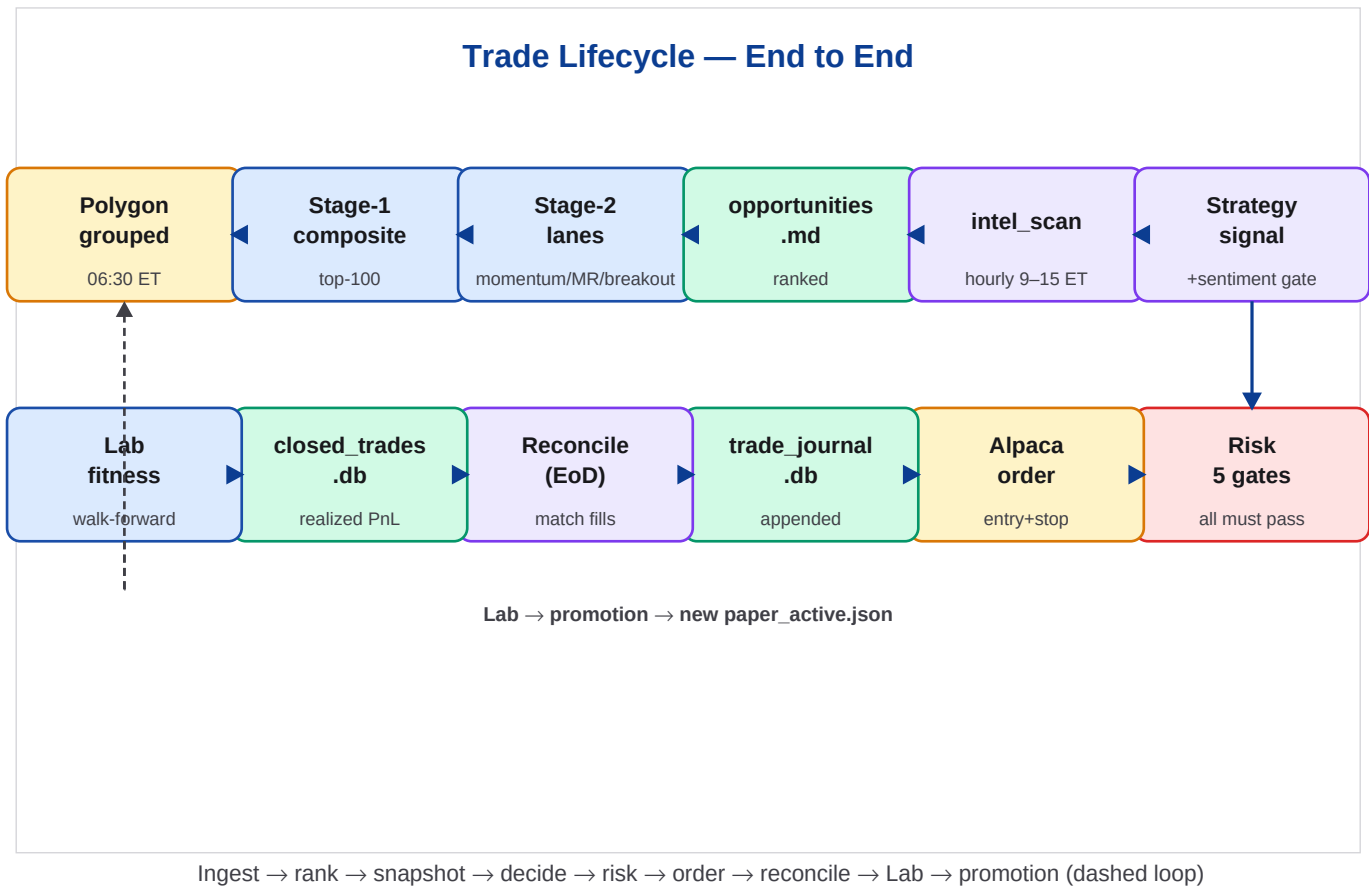
◆ Why a role architecture?

Failure isolation — a bad Polygon response in `SentimentAnalyst` can't kill `StockScanner`. **SLA tracking** — every role declares `sla_seconds`; latency is recorded for the dashboard. **Auditability** — each run produces a JSON record. **Test isolation** — roles are unit-tested with mocked context.

Role	File	What it does	Cadence
HealthPulse	<code>roles/health_pulse.py</code>	Heartbeat to <code>data/heartbeat.json</code> so supervisor detects stale	every 60s
StockScanner	<code>roles/stock_scanner.py</code>	Runs <code>intel_scan</code> over Stage-2 watchlist; emits BUY/HOLD/SELL if <code>fallback</code> flag set.	every 15 min, M–F
CryptoScanner	<code>roles/crypto_scanner.py</code>	24/7 scan of BTC/USD, ETH/USD, SOL/USD	every 30 min, 24/7
PortfolioMonitor	<code>roles/portfolio_monitor.py</code>	Reconciles Alpaca positions vs <code>state.db</code> ; alerts on drift	9–16 ET hourly
OrderSteward	<code>roles/order_steward.py</code>	Verifies every open order has a valid stop; kills orphans	9–16 ET hourly
SentimentAnalyst	<code>roles/sentiment_analyst.py</code>	Pre-warms Polygon news sentiment cache	08:55 + 12:00
UniverseCurator	<code>roles/universe_curator.py</code>	Refreshes Polygon grouped (06:30); ranks (07:30); midday rank (12:00)	06:30 + 07:30 + 12:00
VipListener	<code>roles/vip_listener.py</code>	Tracks watched handles. Phase 0 = alert-only.	9–16 ET, every 30 min
Reporter	<code>roles/reporter.py</code>	Emails midday and EoD HTML digests	12:31 + 18:00
StrategyCoach	<code>roles/strategy_coach.py</code>	<i>Phase 4.</i> 30d alpha vs SPY; flips fallback flag with 0.50, 0.45	09:30, M–F
HoldSpyCoordinator	<code>roles/hold_spy_coordinator.py</code>	<i>Phase 4.</i> Drives 5-day SPY transition when fallback flips	15:15, M–F
StrategyArchitect	<code>roles/strategy_architect.py</code>	<i>Phase 5.</i> Calls Claude API for parameter proposals	Lab-driven
CodeReviewer	<code>roles/code_reviewer.py</code>	<i>Phase 5.</i> Calls Claude API for code review	Lab-driven

4. The Trade Lifecycle

Six stages in a continuous loop — from raw Polygon bytes to a reconciled trade in `closed_trades.db` that feeds the Lab back to the strategy.



Stage 1 — Market Data Ingestion

06:30 ET (massive_refresh) — UniverseCurator pulls Polygon's `/v2/aggs/grouped/locale/us/market/stocks/{date}` via `massive_client.py:daily_grouped()`. All US equities for one trading day in a single call. Result lands in `data/massive_grouped.db`.

INFO

If Polygon is unavailable or `POLYGON_API_KEY` is missing, the system falls back to `_CORE_LIQUID_TICKERS_RAW` intersected with Alpaca's tradable list — universe never goes empty.

08:55 + 12:00 ET (news_warm_*) — SentimentAnalyst loops the active universe; per-article sentiments (positive=+1, neutral=0, negative=-1) averaged per symbol-day, written to `news_sentiment.db`.

Stage 2 — Universe Ranking

Stage-1: Composite Score (07:30 ET)

`screener.py:build_stage1_shortlist()` — fast, broad, deliberately simple:

```
score = 1d_return * 1.4
```

```
+ (5d_return - SPY_5d_return)
+ min(volume_ratio, 3) * 2
```

Top 100 survive. Each becomes a RankedCandidate with symbol, score, last price, 5d %, vol ratio, sector tags.

Stage-2: Strategy Lanes (parallel, ThreadPoolExecutor)

- **MomentumLane** — $RSI \in [55, 70]$, $MACD > \text{signal}$, $\text{price} > \text{EMA}_{20}$, $5d > 0$.
- **MeanReversionLane** — $RSI < 30$, $\text{price} < \text{lower Bollinger}$.
- **BreakoutLane** — $\text{close} > 20\text{-day high}$, $\text{volume} \geq 3 \times \text{avg}$.

Multi-lane endorsements for the same symbol are merged: max conviction wins; lanes and reasons concatenate. Result is sorted MergedCandidate rows.

Stage 3 — Snapshot to opportunities.md

Instead of a DB table, the ranked list is written to a Markdown file. Today's snapshot:

```
# Opportunities (Stage-2)
Generated: 2026-04-27T12:27:14+00:00
Total endorsed: 14

## Lane Counts
- momentum: 10
- mean_reversion: 2
- breakout: 3

### 1. ARB/USD (CRYPTO)
- Lanes: breakout
- Conviction: 0.90
- Stage-1 score: 27.06
- Last price: $0.1019445
- Why: Close $0.13 > 20d high $0.12, vol 4.6x avg
```

* NOTE

This file is the contract between the curator and every downstream consumer (StockScanner, Lab, midday rerank). Git-friendly, human-inspectable, trivially editable.

Stage 4 — Decision & Order Placement

StockScanner.safe_run() calls cli.intel_scan() once per cycle. The internal decision flow is shown on the next page. For BUY signals:

- **Entry order** — market, day, last_close.
- **Stop-loss order** — stop-limit, GTC. Crypto adds 5% buffer between stop and limit.
- Both order IDs persist to trade_journal.db.

Stage 5 — Reconciliation & PnL

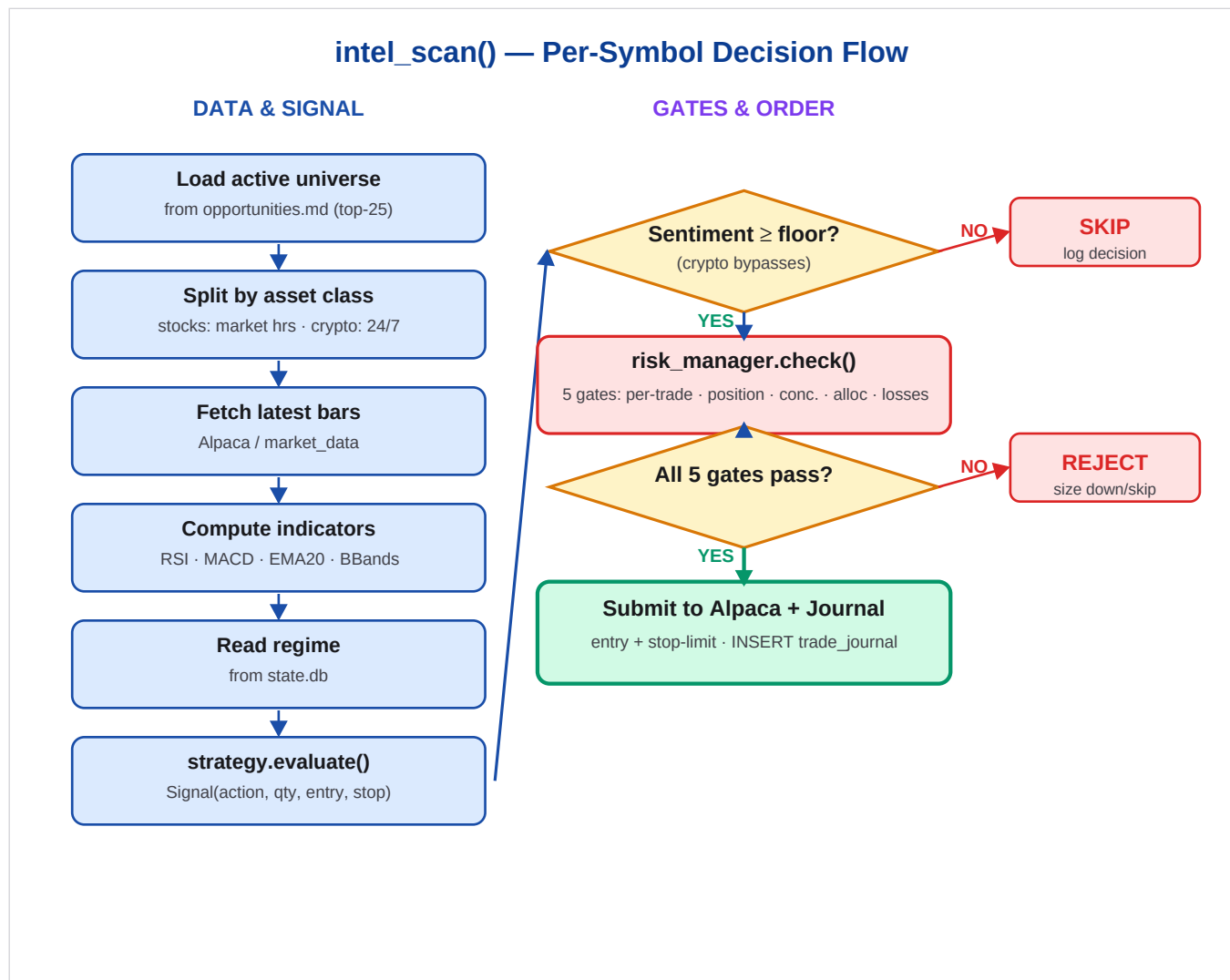
PortfolioMonitor reconciles Alpaca positions vs state.db every hour; OrderSteward verifies stops are still attached. End-of-day reconciliation.py matches exit fills against entries (by symbol+order_id), computes $\text{realized_pnl} = (\text{exit} - \text{entry}) \times \text{qty}$ and hold_hours, and appends to closed_trades.db.

Stage 6 — Lab Feedback

Lab reads closed_trades.db, runs walk-forward backtests on parameter variations, scores fitness (alpha vs SPY, Sortino, max DD). High-fitness proposals can be promoted — data/paper_active.json is rewritten with the new parameters and a fresh version + git_sha + fitness_at_promotion stamp.

5. Decision Flow inside intel_scan

The branching logic that decides whether a single symbol becomes an order. Two decision diamonds gate execution: the **sentiment gate** (skipped for crypto) and the **5-gate risk check**.



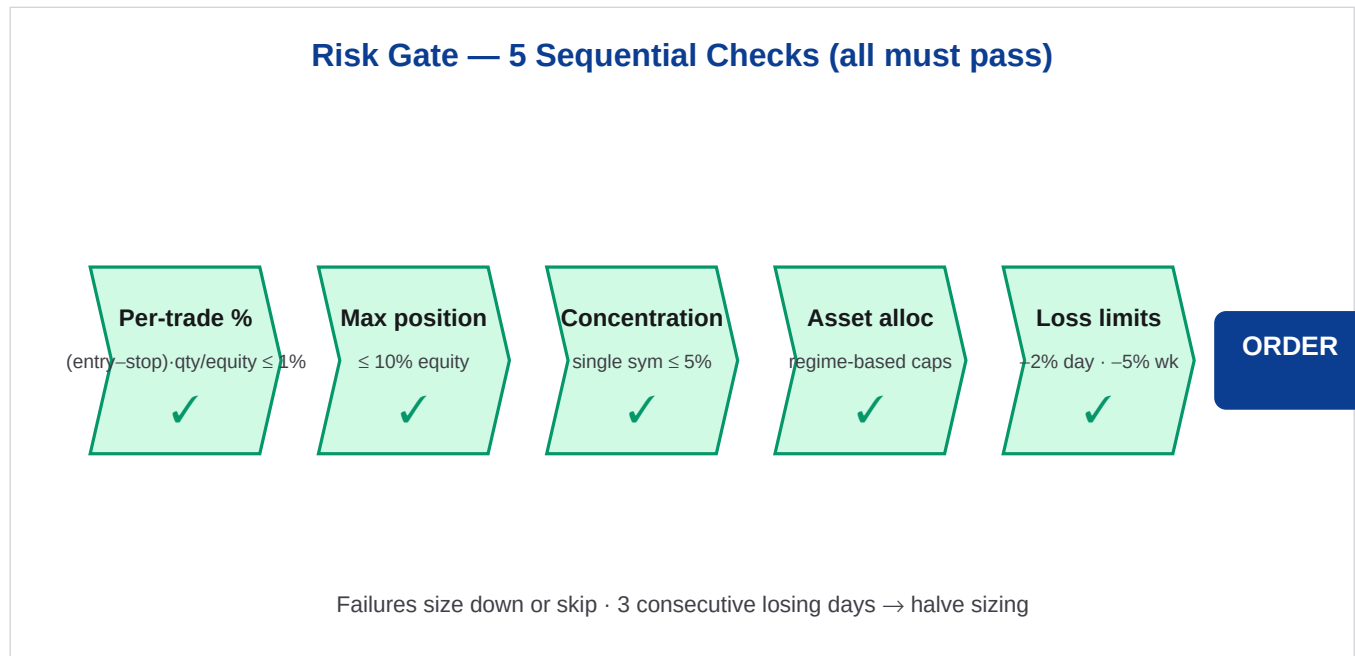
Per-symbol flow inside intel_scan() — green path = order placed, red = skip/reject

■ Sentiment gate logic

If the cached score for this symbol is $< \text{sentiment_floor}$ (currently -0.5), the BUY is skipped. Crypto bypasses this gate (Polygon news is equity-focused). Symbols with no cached news data **pass** by default — coverage gaps don't block entries.

6. Risk Gate — 5 Sequential Checks

Every signal that survives the sentiment gate must pass all five risk checks in `risk_manager.check()`. The defaults below come from `strategy/config.yaml`:



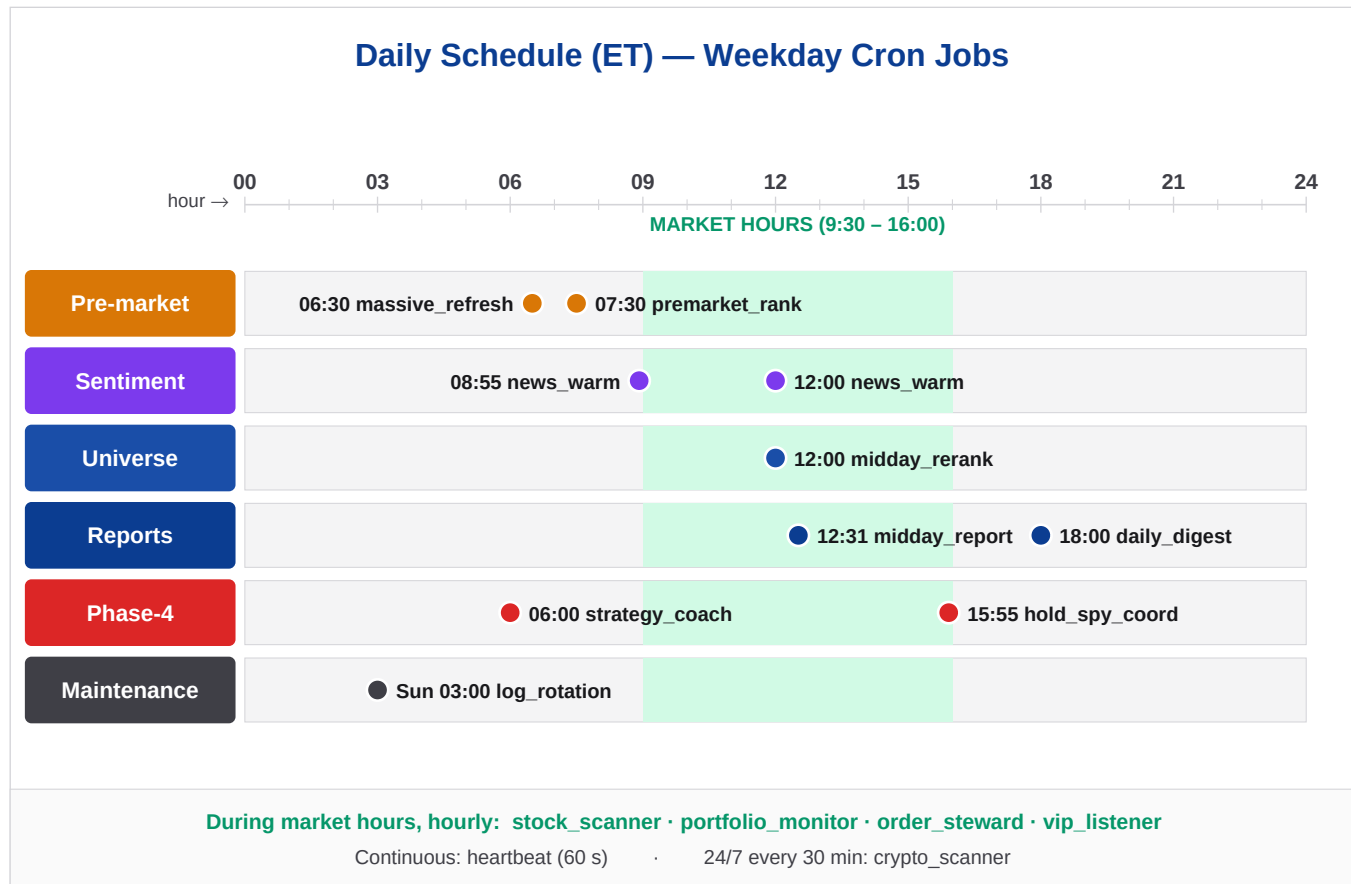
#	Gate	Default Limit	Source
1	Per-trade risk %	1.0%	config.yaml: risk.per_trade_risk_pct
2	Max position %	10%	config.yaml: risk.max_position_pct
3	Symbol concentration %	5%	config.yaml: risk.max_symbol_concentration_pct
4	Asset-class allocation	regime-based	config.yaml: regime_allocations
5	Daily / weekly losses	-2% · -5%	config.yaml: risk.daily/weekly_loss_limit_pct

i INFO

Per-trade math. $(\text{entry} - \text{stop}) \times \text{qty} / \text{equity}$ must be $\leq 1\%$. If it exceeds, qty is reduced or trade is skipped. After 3 consecutive losing days, sizing is automatically halved.

7. Scheduling — Daily Timeline

Every recurring action lives in `scheduler_jobs.py`. Cadence values come from `data/paper_active.json::cadence` with fallbacks in `cadence.py`.



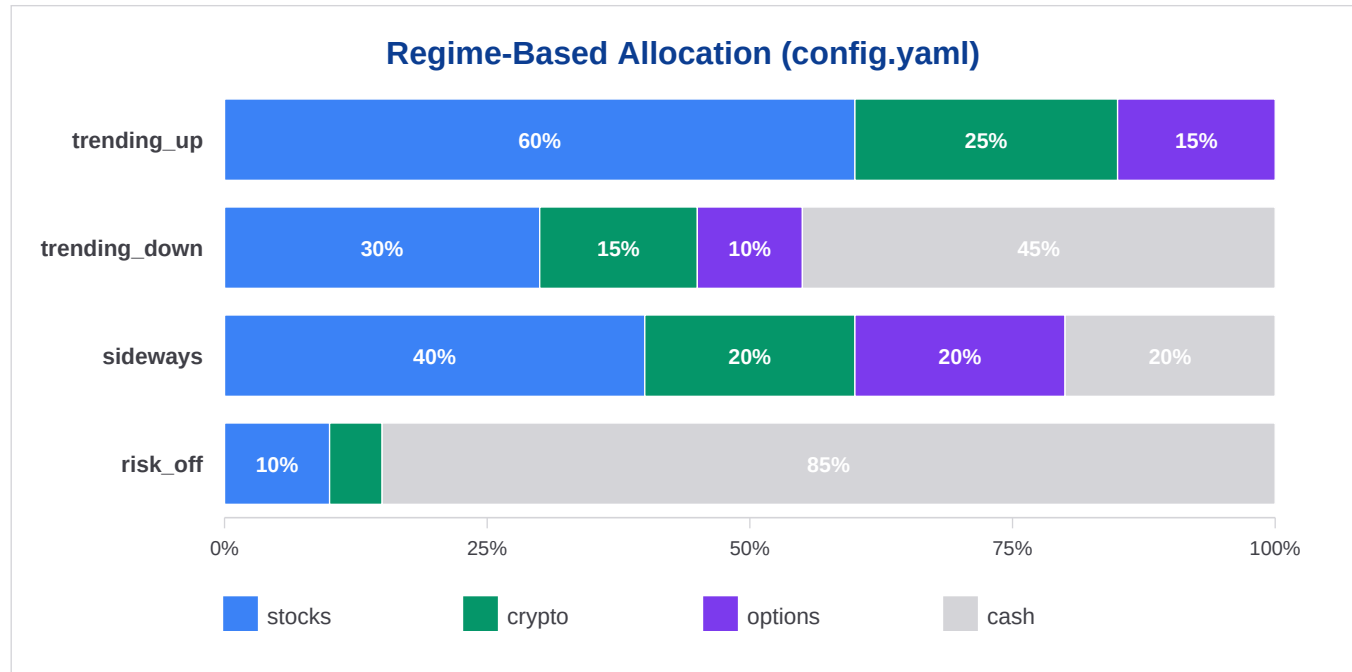
Green band = market hours. Hourly jobs (`stock_scanner`, `portfolio_monitor`, `order_steward`, `vip_listener`) run at the top of every market hour.

Job ID	Trigger	Time (ET)	Purpose
heartbeat	Interval	every 60s	Liveness pulse for supervisor
stock_scanner	Cron	9–15 hourly, M–F	intel_scan over Stage-2 watchlist
crypto_scanner	Interval	every 30m, 24/7	Scan BTC/ETH/SOL pairs
portfolio_monitor	Cron	9–16 hourly, M–F	Reconcile positions
order_steward_sweep	Cron	9–16 hourly, M–F	Verify stops; kill orphans
vip_listener	Cron	9–16 every 30m, M–F	Watched-handle alerts
news_warm_morning	Cron	08:55, M–F	Pre-warm sentiment cache
news_warm_midday	Cron	12:00, M–F	Mid-day sentiment refresh
massive_refresh	Cron	06:30, M–F	Pull Polygon grouped
premarket_rank	Cron	07:30, M–F	Stage-1/2 + write opportunities.md

Job ID	Trigger	Time (ET)	Purpose
midday_rerank	Cron	12:00, M–F	Re-rank before 12:30 scan
midday_report	Cron	12:31, M–F	HTML midday digest
daily_digest	Cron	18:00, M–F	EoD email summary
log_rotation	Cron	Sun 03:00	Archive runs/*.json
strategy_coach	Cron	06:00, M–F	Phase-4 alpha-vs-SPY check
hold_spy_coordinator	Cron	15:55, M–F	Phase-4 SPY transition driver

8. Regime-Driven Allocation

`regime.py` classifies each day into one of four buckets from VIX, SPY 5-day return, and 20-day realized volatility. The regime determines which strategy is selected and how capital is split. Caps come straight from `strategy/config.yaml`:



In `risk_off` the book moves to 85% cash; in `trending_up` nearly all capital deploys.

■ Regime triggers

trending_up · SPY 5d > 0 and 20d vol below threshold · **trending_down** · SPY 5d < 0 and trend confirmed · **sideways** · low directional move · **risk_off** · 20d realized vol > 22% (`config.yaml`: `regime.vol_threshold_pct`).

9. External Integrations & APIs

Service	Endpoint	Used For	Auth
Alpaca Paper	paper-api.alpaca.markets/v2	Orders, positions, equity	ALPACA_API_KEY/SECRET
Polygon	api.polygon.io	Grouped bars, news, short interest	POLYGON_API_KEY
FRED	fredgraph.csv	VIX, 10Y yield, fed funds	Anonymous
GDELT 2.0	gdeltproject.org	Global news event sentiment	Anonymous
SEC EDGAR	sec.gov RSS	Insider Form 4 filings	Anonymous
Anthropic	api.anthropic.com/v1	Phase 5+ proposals, code review	ANTHROPIC_API_KEY
Gmail SMTP	smtp.gmail.com:587	Daily digest, midday, alerts	EMAIL_APP_PASSWORD

* NOTE

No MCP servers in this repo today. Phase 5 LLM features call `anthropic_client.py` directly. Memory notes indicate a future `trading-intelligence-mcp` is planned to wrap SEC, GDELT, FRED, Alpaca, and public news.

10. Configuration, Secrets & Safety Rails

Where settings live

File	Purpose	Loaded By
strategy/config.yaml	Risk caps, regime allocations, sentiment floor, email targets	config.py:load_config()
data/paper_active.json	Versioned snapshot — params, cadence, universe, risk, fitness, git sha, promoted_at	state.py:load_snapshot()
ops/paper_active.template.json	Baseline before first promotion	ops/install.sh
strategy/watchlist.yaml	Static fallback universe	state.py:load_watchlist()
strategy/opportunities.md	Live ranked Stage-2 list (rewritten each cycle)	orchestrator.py:load_ranked_watchlist()
strategy/vip_handles.yaml	Watched social/news handles	VipListener role
.env (uncommitted)	All secrets — Alpaca, Polygon, Anthropic, Gmail	python-dotenv at process start

Active strategy snapshot (today)

Current contents of data/paper_active.json (promoted 2026-04-28 by lab-promoter, fitness 3.97):

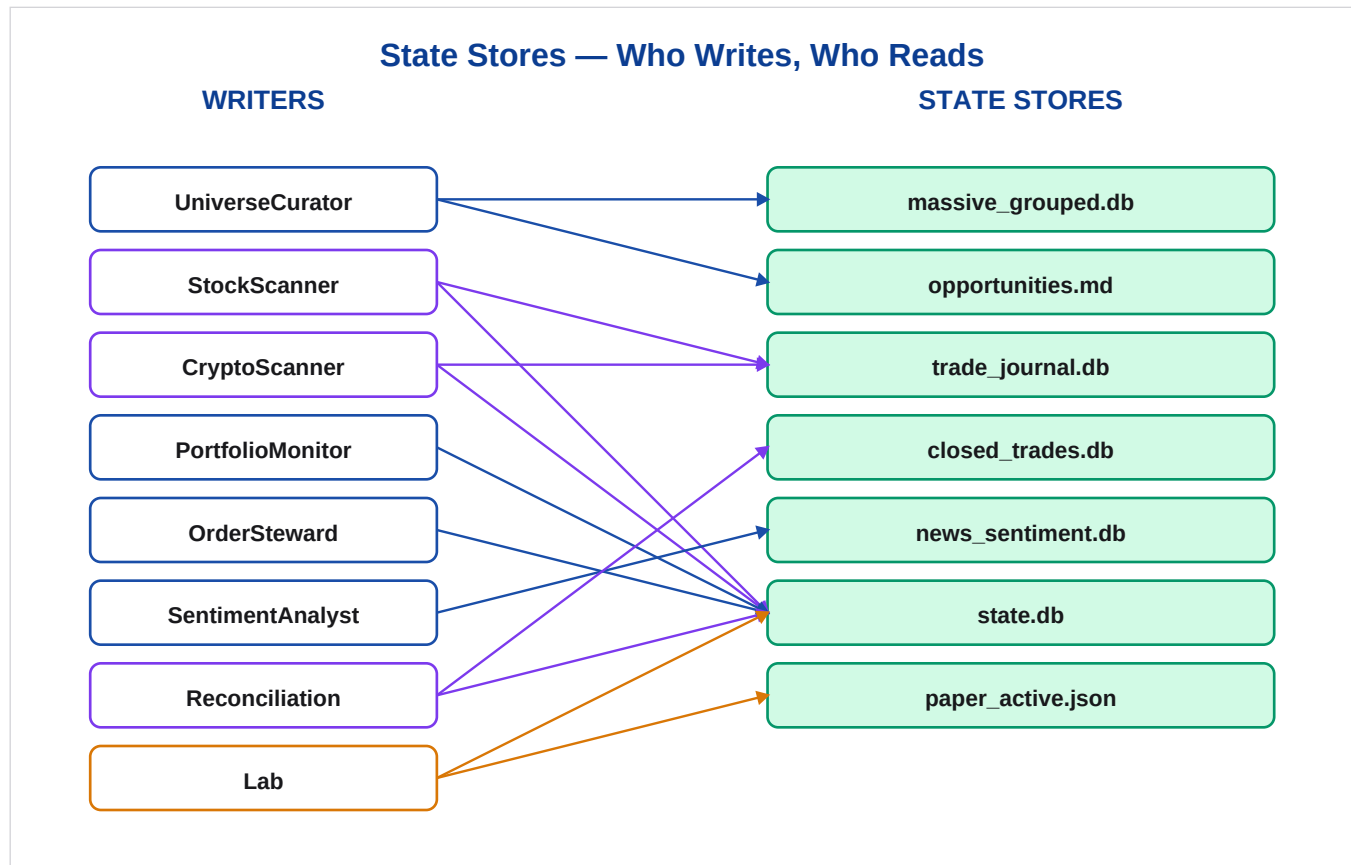
```
{
  "version": "auto-20260428-100154",
  "git_sha": "phase1-bootstrap",
  "active_template": "momentum",
  "fitness_at_promotion": 3.967,
  "fallback_when_no_alpha": "hold_spy",
  "params": { "ema_period": 16, "rsi_lower": 50.077,
    "rsi_upper": 70.366, "stop_pct": 6.110,
    "sentiment_floor": -0.158 },
  "risk_caps": { "daily_loss_pct": 3, "max_drawdown_pct": 20,
    "max_position_pct": 10 },
  "universe": { "stocks_filter": "stage1_top100",
    "crypto_pairs": ["BTC/USD", "ETH/USD", "SOL/USD"] },
  "cadence": { "heartbeat_seconds": 60,
    "stock_scanner_minutes": 60, ... }
}
```

Secrets in .env

- **ALPACA_API_KEY / ALPACA_API_SECRET** — paper credentials.
- **ALPACA_BASE_URL** — must start with `https://paper-api.alpaca.markets`.
- **POLYGON_API_KEY** — required for grouped bars + Polygon news.
- **ANTHROPIC_API_KEY** — Phase 5 LLM evolution.
- **GMAIL_USER / GMAIL_APP_PASSWORD** — daily / midday emails.
- **BOT_MODE** — must equal paper; live raises LiveModeDisabled.

11. State Stores — Who Writes, Who Reads

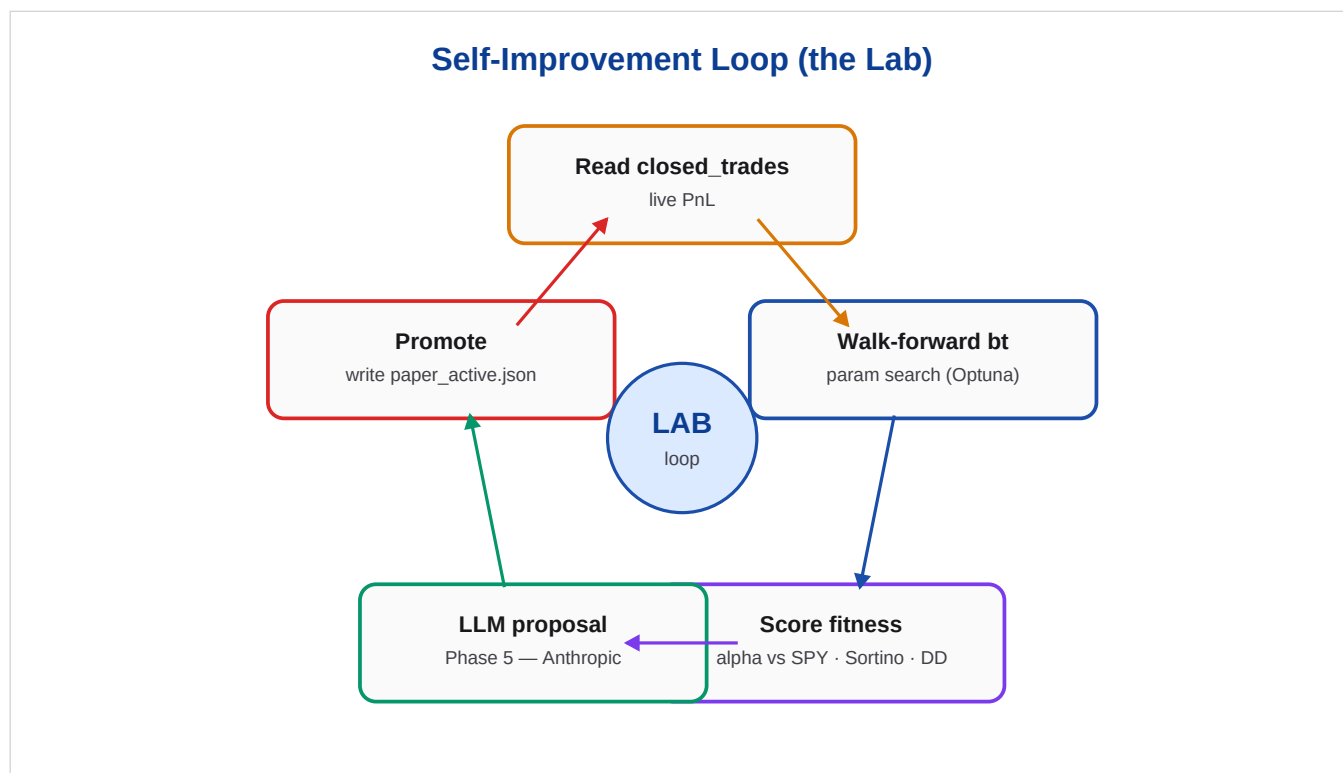
All state is local. Nothing requires an external database. SQLAlchemy 2.0 ORM, SQLite engine, WAL mode on state.db for concurrent reads. Alembic migrations auto-applied at boot.



Database / File	Schema	Purpose
state.db	24+ ORM tables: Heartbeat, RoleRun, RoleKpi, RegimeHistory, ConfigHistory, Leaderboard, FallbackFlag, TemplateProposal, AnthropicCostLog, ...	Central coordination · WAL mode
trade_journal.db	trades — entry signals + order IDs	Append-only entry log
closed_trades.db	closed_trades — entry/exit, realized_pnl, hold_hours	Drives Lab fitness
news_sentiment.db	(symbol, date) → score	Entry-gate input
massive_grouped.db	Polygon daily OHLC for full universe (~10 MB)	Polygon cache
paper_active.json	Active strategy snapshot	What the daemon runs
heartbeat.json	Last daemon pulse	Stall detection
pause.flag	Optional file	Manual kill switch
runs/*.json	Per-role per-day JSON	Audit trail

12. Lab — Self-Improvement Loop

The Lab is what makes the system *self-evolving*. It runs in its own launched process and continuously cycles through five phases:



The Lab pulls live results, searches parameter space, scores fitness, and — when it beats the current strategy — promotes a new paper_active.json.

◆ Walk-forward, not in-sample

The Lab uses **walk-forward analysis** (overlapping windows: e.g. 30-day train, 5-day test, rolling forward). Optuna searches parameter space within each window; **average out-of-sample fitness** is what gets promoted, not in-sample performance.

\$ LLM cost discipline

Every Anthropic API call is logged to `state.db::AnthropicCostLog` with token counts. `cost_tracker.py` halts the LLM loop if daily spend exceeds threshold. Trading itself is unaffected.

13. Operations & Failure Modes

launchd plists

Plist	Process	Purpose
com.bharath.trading.daemon.paper	python -m trading_bot.daemon	Main trading daemon
com.bharath.trading.lab	python -m trading_bot.lab	Walk-forward evolution
com.bharath.trading.supervisor	supervisor	Health monitor / alerts
com.bharath.trading.daemon.live	(disabled)	Phase 6 scaffolding

Boot sequence

- Load .env via python-dotenv.
- Validate BOT_MODE=paper.
- Validate ALPACA_BASE_URL is paper.
- Run Alembic migrations against state.db.
- Load paper_active.json → cadence, params, risk caps.
- Construct roles, register all jobs via scheduler_jobs.register_jobs().
- Install SIGTERM handler.
- Enter APScheduler event loop.

Failure modes

Failure	Detection	Behavior
Polygon outage / no key	MassiveAuthError or empty response	Fallback to _CORE_LIQUID_TICKERS_RAW. Universe never empty.
Alpaca outage	HTTP error in alpaca_client	Order skipped; PortfolioMonitor flags drift next cycle.
Sentiment cache miss	news_sentiment lookup returns None	passes_filter() returns True — entry not blocked.
Daemon stall	Supervisor reads heartbeat.json — stale	Email alert; launchd auto-restarts.
Daily loss > -2%	risk_manager check at every signal	All new entries blocked for the day.
3 consecutive losing days	pnl_state aggregation	Position sizing automatically halved.
30-day alpha < SPY	StrategyCoach 06:00 ET	FallbackFlag set; HoldSpyCoordinator drives 5-day SPY transition.
LLM daily spend exceeded	cost_tracker before each call	Lab evolution halted; trading unaffected.
Manual stop	touch data/pause.flag	Order-placing jobs skip; monitoring continues.

14. Worked Example: One Trade

Following **ARB/USD** from today's opportunities.md (rank #1, conviction 0.90) through the entire pipeline:

14.1 How it got onto opportunities.md

```
06:30 ET → UniverseCurator.run_refresh()
    Polygon /v2/aggs/grouped → daily bars cached to massive_grouped.db

07:30 ET → UniverseCurator.run_rank()
    Stage-1: ARB/USD score = 27.06 (high vol_ratio + 5d outperformance)
        → makes top-100
    Stage-2 (BreakoutLane):
        close $0.13 > 20d high $0.12 ✓
        volume = 4.6× average ✓
        → emits LaneCandidate(ARB/USD, conviction=0.90, lane=breakout)
    screener.write_opportunities_snapshot()
        → ARB/USD lands at rank #1 in strategy/opportunities.md
```

14.2 Decision (next intel_scan tick)

```
scheduler fires intel_scan (e.g. 10:00 ET)
    StockScanner.safe_run() → cli.intel_scan.callback()

    _load_active_universe() → top-25 from opportunities.md (ARB/USD ✓)
    asset_class = CRYPTO → bypass sentiment gate
    fetch latest crypto bars from alpaca_client
    compute RSI, MACD, EMA20, Bollinger
    regime = trending_up → MomentumStrategy.evaluate()
        → Signal(BUY, qty=1000, entry=$0.1019, stop=$0.0965)
```

14.3 Risk & order

```
risk_manager.check(signal):
    per_trade_risk = ($0.1019 - $0.0965) × 1000 = $5.40 (~0.5%) ✓
    max_position   = $0.1019 × 1000 = $101.90 (1% of equity) ✓
    concentration  = 1% ≤ 5% ✓
    asset-class alloc (regime=trending_up, crypto cap=25%) ✓
    daily/weekly loss limits ✓

alpaca_client.submit_order(BUY ARB/USD):
    POST /v2/orders → market entry, qty=1000 → order_id 'abc123'
    POST /v2/orders → stop-limit (stop=$0.0965, limit=$0.0917)
                        → order_id 'def456'

trade_journal.append():
    INSERT trades (symbol='ARB/USD', side='buy', qty=1000,
        entry_order_id='abc123', stop_loss_order_id='def456')
```

14.4 Tracking & exit

```
PortfolioMonitor (next hour):
    Alpaca says ARB/USD pos size 1000, mark $0.105, +3% unrealized

OrderSteward (next hour):
    stop-limit 'def456' still attached on correct side ✓

(later) stop fires → entry-side cancelled; closing fill recorded

EoD reconciliation.py:
    match exit fill against trade_journal entry by symbol+order_id
```

```
realized_pnl = ($0.103 - $0.1019) × 1000 = $1.10  
hold_hours   = 4.5  
INSERT closed_trades(...)
```

Lab (overnight):

```
reads closed_trades, computes fitness  
if a new param set beats current → propose to TemplateProposal  
promoter (next cycle) → write new paper_active.json
```

15. Quick Reference

Where to look for...

You want to know about...	Open this file
Order submission and Alpaca	src/trading_bot/alpaca_client.py
The decision loop	src/trading_bot/orchestrator.py
Per-strategy entry rules	strategy.py + strategy_lanes.py
Risk gate (5 checks)	risk_manager.py
Universe ranking (Stage-1/2)	screener.py + universe.py
Polygon access	massive_client.py
Macro snapshot (FRED/GDELT/SEC)	intelligence.py
Sentiment cache + entry gate	news_sentiment.py
All scheduled jobs and timing	scheduler_jobs.py
Daemon boot sequence	daemon.py
Trade ledger	trade_journal.py
Reconciliation + closed trades	reconciliation.py
P&L and equity HWM	pnl_state.py
Regime classification	regime.py
State / DB schema	state_db.py
Lab loop (parameter search)	lab.py + walkforward.py
LLM evolution (Phase 5)	anthropic_client.py + roles/strategy_architect.py
Dashboard	dashboard/app.py
Email reports	reports.py + email_sender.py
Live ranked universe	strategy/opportunities.md
Active strategy snapshot	data/paper_active.json
Risk knobs & allocations	strategy/config.yaml

Common operations

- **Pause trading:** touch data/pause.flag
- **Resume trading:** rm data/pause.flag
- **Inspect today's universe:** open strategy/opportunities.md
- **Inspect last scan:** open data/last_scan.json
- **Manual scan:** bot intel-scan
- **Manual rank refresh:** bot rank

- **Promote a Lab proposal:** `bot promote`
- **Roll back strategy:** restore an older `paper_active.json` from git, restart daemon
- **Restart daemon:** `launchctl unload && load com.bharath.trading.daemon.paper.plist`

End of architecture document. For deeper component reads, jump to the files cited above — this PDF is a map, not the territory.